Wydział
Budowy Maszyn i Informatyki

Uniwersytet
Bielsko-Bialski

Technologie Blockchain

SPRAWOZDANIE

Laboratorium nr 1

Temat: Środowisko programistyczne PyCharm i prototypowy szkielet blockchaina.

KOŁODZIEJCZYK KAROLINA
Kierunek: Cyberbezpieczeństwo II stopnia
Semestr: I zaoczny
Grupa: 2a

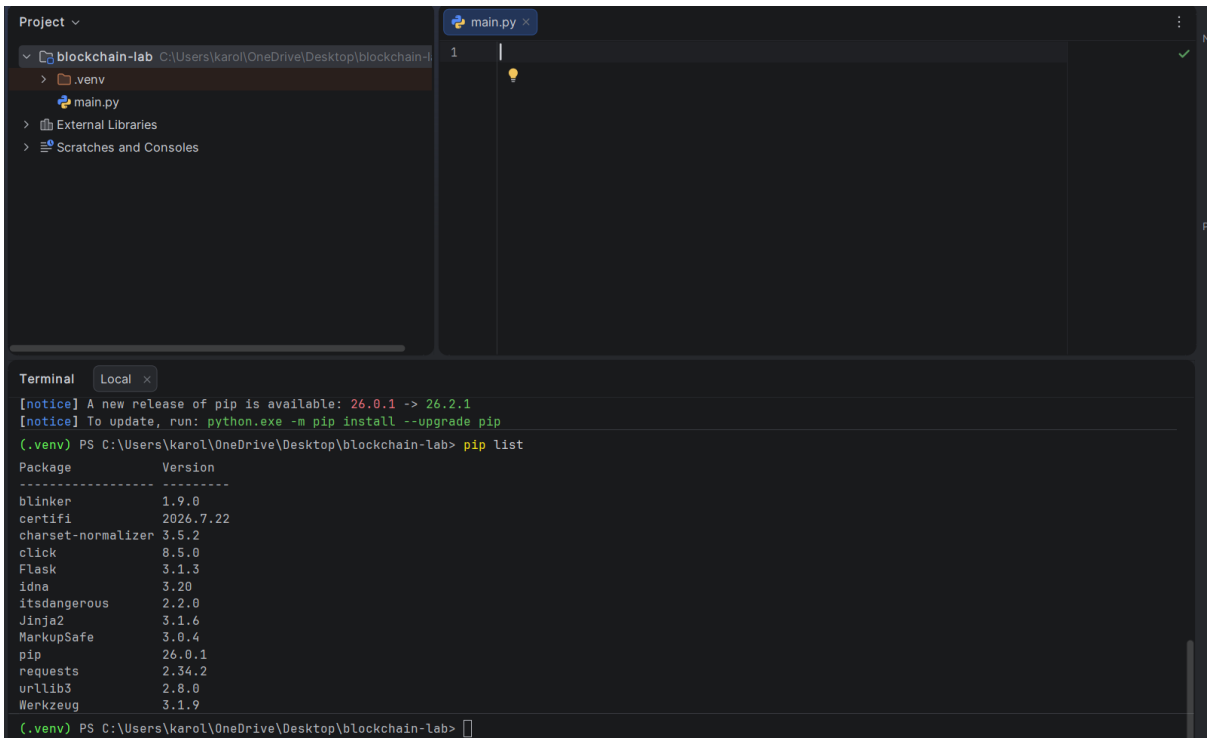
Link: https://github.com/ryzykotolerowalne/TB_lab_sem1

Cel ćwiczenia:

Przygotowanie środowiska programistycznego PyCharm oraz zbudowanie w języku Python prototypowego szkieletu łańcucha bloków, pozwalającego zrozumieć rolę funkcji skrótu i powiązań między blokami w zapewnianiu integralności danych.

Opis ćwiczenia:

W pierwszej kolejności przeanalizowano punkt 1. Wprowadzenie – przeanalizowano czym jest Blockchain i jak wygląda budowa łańcucha bloków. Po wnikliwej lekturze, można było przejść do wykonania konkretnego ćwiczenia laboratoryjnego. W programie pycharm został utworzony projekt o stosownej nazwie oraz zostały zainstalowane odpowiednie pakiety.



W dalszej części ćwiczenia zaimplementowano klasę Blockchain zgodnie z przykładem 3 i dodano do łańcucha co najmniej 3 bloki z przykładowymi transakcjami.

```python
if __name__ == "__main__":
    bc = Blockchain()

    bc.add_block(["Alice -> Bob: 5"])
    bc.add_block(["Bob -> Carol: 2"])


    bc.add_block(["Carol -> Alice: 3"])
    bc.add_block(["Bob -> Carol: 1"])
    bc.add_block(["Alice -> Bob: 7"])

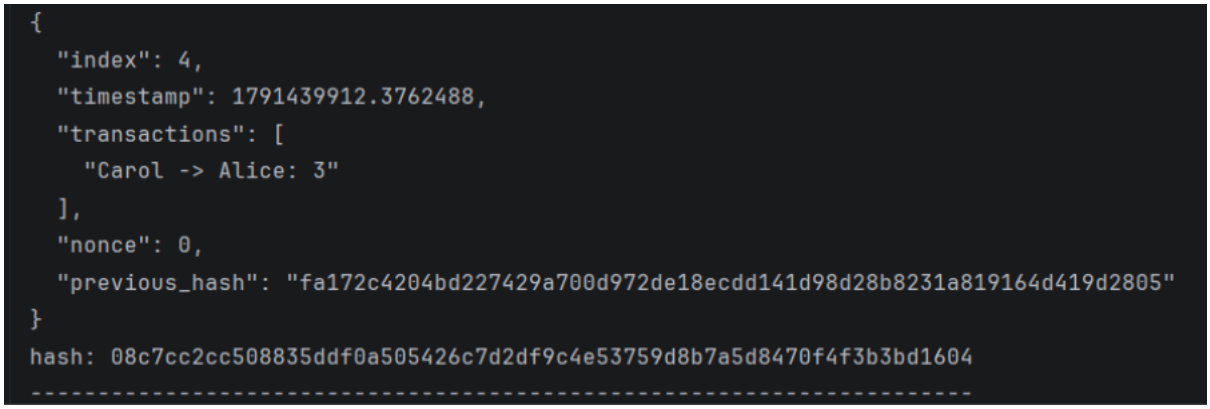
```

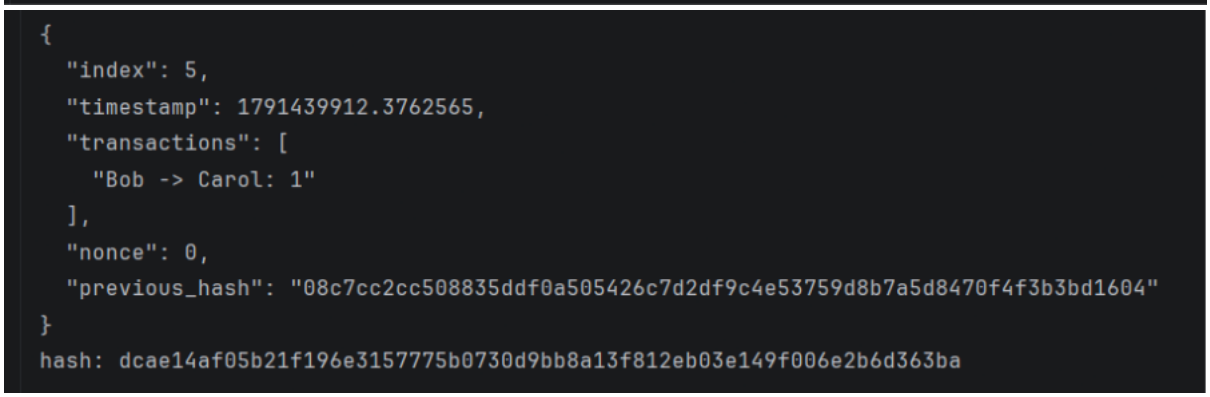
Wyświetlono zawartość łańcucha wraz ze skrótami bloków i sprawdzono, czy pola previous_hash wskazują na właściwe bloki. Poniższe zrzuty tego dowodzą

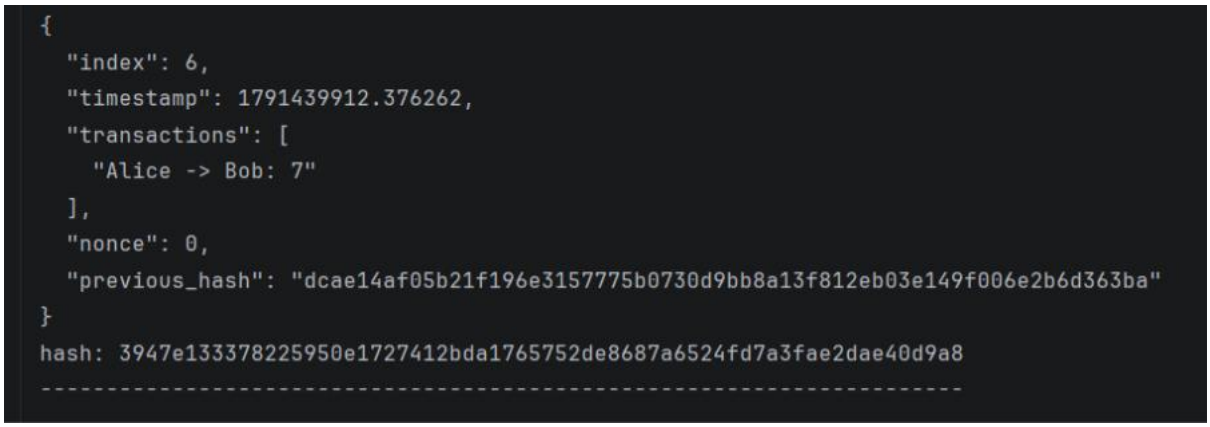

```
{
  "index": 1,
  "timestamp": 0,
  "transactions": [],
  "nonce": 0,
  "previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
--------------------------------------------------------------------
```

```
{
  "index": 2,
  "timestamp": 1791439912.376171,
  "transactions": [
    "Alice -> Bob: 5"
  ],
  "nonce": 0,
  "previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172"
}
hash: 239c31caaa01f09907cb60e1abbb5a2afd93d4dd8479cade66556c5c4c43892c
--------------------------------------------------------------------
```

```
{
  "index": 3,
  "timestamp": 1791439912.3762302,
  "transactions": [
    "Bob -> Carol: 2"
  ],
  "nonce": 0,
  "previous_hash": "239c31caaa01f09907cb60e1abbb5a2afd93d4dd8479cade66556c5c4c43892c"
}
hash: fa172c4204bd227429a700d972de18ecdd141d98d28b8231a819164d419d2805
--------------------------------------------------------------------
```

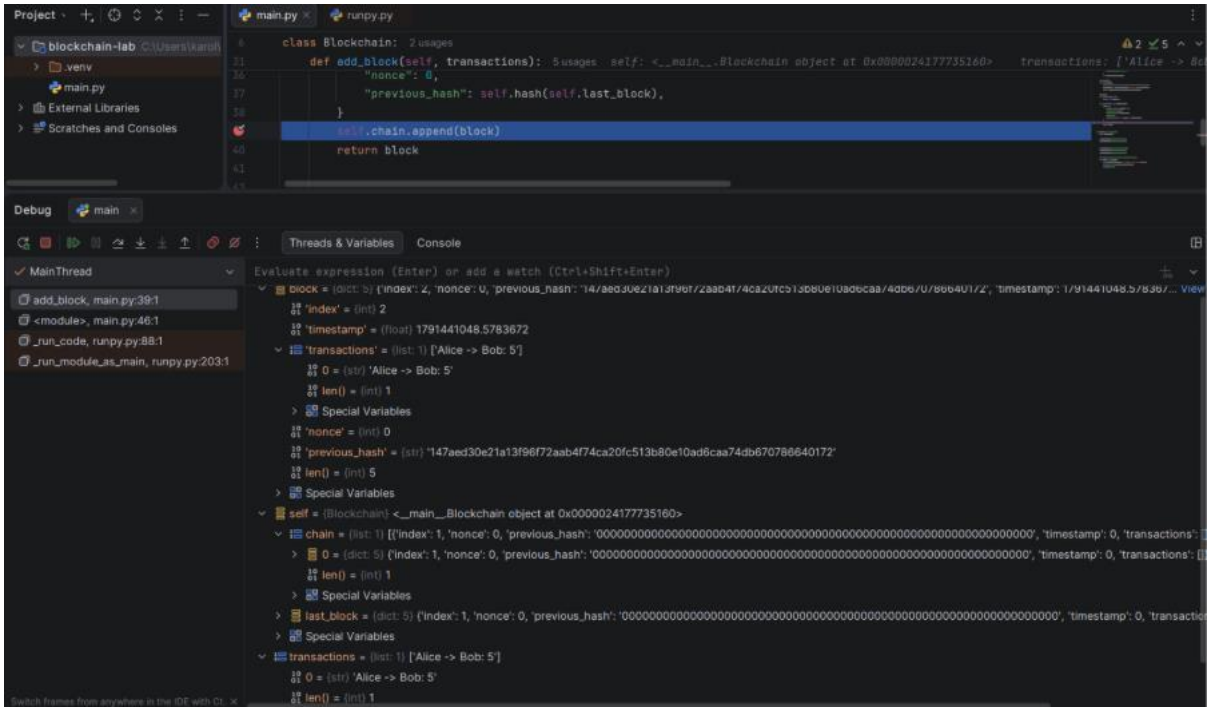






Ustawiono punkt przerwania w metodzie add_block() i prześledzono w debuggerze tworzenie nowego bloku.

Zrzut ekranu okna Variables dołączono do sprawozdania.



Kolejnym etapem było sprawdzenie doświadczalnie, jak zmieni się skrót bloku, jeżeli w metodzie hash() usunięty zostanie parametr sort_keys=True i zmieni się kolejność pól w słowniku bloku.

```
@staticmethod  2 usages
def hash(block):
    # sort_keys=True gwarantuje deterministyczną kolejność pól - nic nie zagwarantuje, bo usunęłam
    block_string = json.dumps(block).encode()
    return hashlib.sha256(block_string).hexdigest()
```

```
31      def add_block(self, transactions):  5 usages
32          block = {
33              "nonce": 0,  # <- po zamianie
34              "index": len(self.chain) + 1,
35              "timestamp": time(),
36              "transactions": transactions,
37              "previous_hash": self.hash(self.last_block),
38          }
39          self.chain.append(block)
40          return block
```

Otrzymany wynik:


Run
main
C:\Users\karol\OneDrive\Desktop\blockchain-lab\.venv\Scripts\python.exe C:\Users\karol\OneDrive\Desktop\blockchain-lab\main.py
=== ZAWARTOŚĆ ŁAŃCUCHA BLOCKCHAIN ===
{
"index": 1,
"timestamp": 0,
"transactions": [],
"nonce": 0,
"previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 08b341c2db3b45b5fe83dcfe220fef2c1797a9d63c45b637c19390db30d23039
{
"nonce": 0,
"index": 2,
"timestamp": 1791441448.8219182,
"transactions": [
"Alice -> Bob: 5"
],
"previous_hash": "08b341c2db3b45b5fe83dcfe220fef2c1797a9d63c45b637c19390db30d23039"
}
hash: fa620d7ae3e822384f4d969c5b9819503fb8e542ff1061a27c8443704ed490c5
{
"nonce": 0,
"index": 3,
"timestamp": 1791441448.8219764,
"transactions": [
"Bob -> Carol: 2"
],
"previous_hash": "fa620d7ae3e822384f4d969c5b9819503fb8e542ff1061a27c8443704ed490c5"
}
hash: c3da410f5b5523c2e4a09c3acde688d26f176653882c5941d6138dcf9fea43e5


More tool windows
hash: c3da410f5b5523c2e4a09c3acde688d26f176653882c5941d6138dcf9fea43e5
{
"nonce": 0,
"index": 4,
"timestamp": 1791441448.821995,
"transactions": [
"Carol -> Alice: 3"
],
"previous_hash": "c3da410f5b5523c2e4a09c3acde688d26f176653882c5941d6138dcf9fea43e5"
}
hash: b94bfdec45d6fcaa72ef36eb1dc4b7b42b7d20327045845befbe3a2a965d794d
{
"nonce": 0,
"index": 5,
"timestamp": 1791441448.8220022,
"transactions": [
"Bob -> Carol: 1"
],
"previous_hash": "b94bfdec45d6fcaa72ef36eb1dc4b7b42b7d20327045845befbe3a2a965d794d"
}
hash: f1df8a23cbd4a13d2dada1d61c6d929f31e60dd84f9c251036ba877b9f2e474e

```
hash: f1df8a23cbd4a13d2dada1d61c6d929f31e60dd84f9c251036ba877b9f2e474e
--------------------------------------------------------------------
{
  "nonce": 0,
  "index": 6,
  "timestamp": 1791441448.8220072,
  "transactions": [
    "Alice -> Bob: 7"
  ],
  "previous_hash": "f1df8a23cbd4a13d2dada1d61c6d929f31e60dd84f9c251036ba877b9f2e474e"
}
hash: 2348a015642f665fbdc406d7d3598215ebaef388b61905a8c5de135d67f25159
--------------------------------------------------------------------

Process finished with exit code 0
```

Po usunięciu parametru sort_keys=True z metody json.dumps() oraz zmianie kolejności definicji pól w słowniku bloku, wartości skrótów (hashy) wygenerowanych dla poszczególnych bloków uległy całkowitej zmianie, mimo zachowania identycznych danych wejściowych.
Parametr sort_keys=True wymusza, aby biblioteka JSON przed zamianą słownika na tekst posortowała wszystkie klucze alfabetycznie (np. najpierw index, potem nonce, na końcu transactions). Dzięki temu, niezależnie od tego, w jakiej kolejności zapiszemy pola w kodzie, wynikowy ciąg tekstowy (string) wysyłany do funkcji SHA-256 będzie zawsze identyczny (deterministyczny).

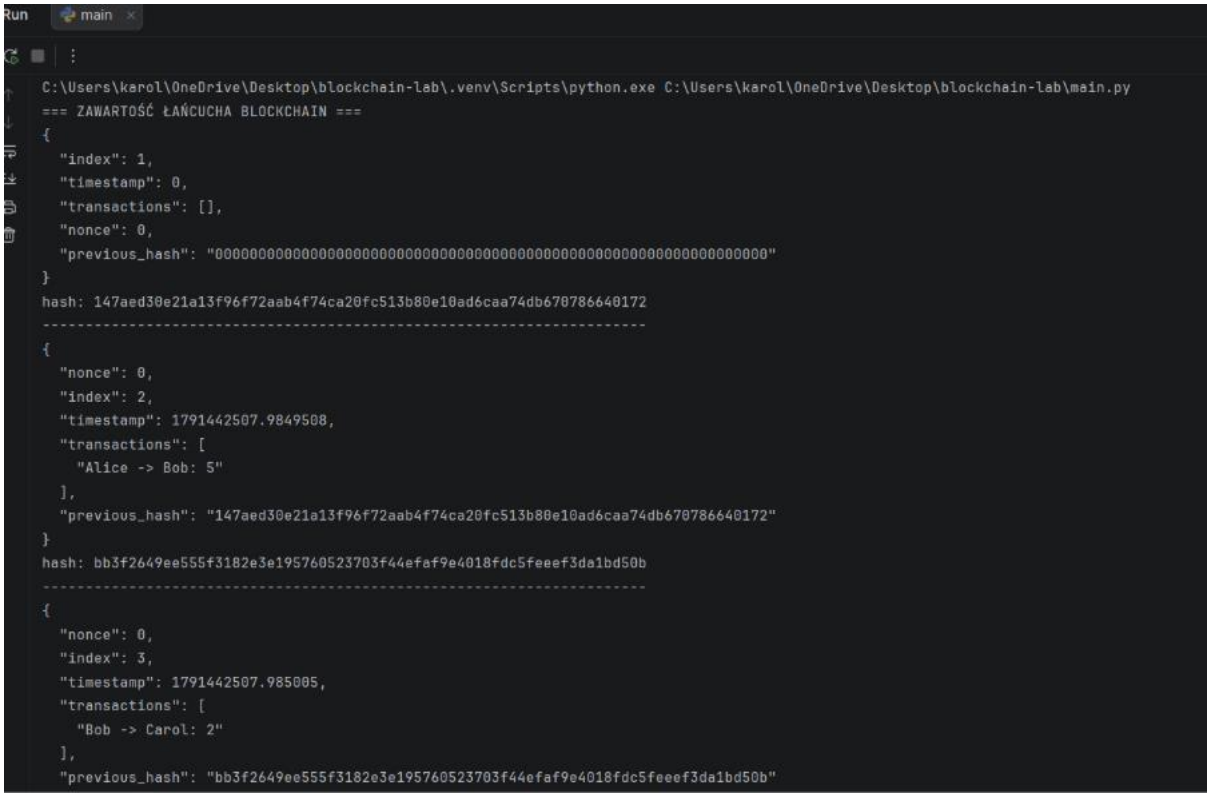
Usunięcie tego parametru sprawia, że zmiana kolejności pól w słowniku generuje zupełnie inny ciąg tekstowy. Ponieważ algorytm SHA-256 charakteryzuje się tzw. *efektem lawinowym*, nawet najmniejsza zmiana w strukturze tekstu (przemieszczenie wyrazów) skutkuje wygenerowaniem całkowicie odmiennego skrótu, Bez sort_keys=True różne komputery w sieci mogłyby generować różne hashe dla tych samych danych, co uniemożliwiłoby osiągnięcie konsensusu.

Kolejnym etapem ćwiczenia była weryfikacja integralności

Zaimplementowano metodę is_chain_valid(), która zwraca informację, czy łańcuch jest poprawny, oraz numer pierwszego niepoprawnego bloku.

```python
    # test integralności

    is_valid, error_block_index = bc.is_chain_valid()
    print(f"Czy łańcuch blockchain jest poprawny? -> {is_valid}")
    print(f"Numer pierwszego błędnego bloku: -> {error_block_index}")
```



```
Run  main

C:\Users\karol\OneDrive\Desktop\blockchain-lab\.venv\Scripts\python.exe C:\Users\karol\OneDrive\Desktop\blockchain-lab\main.py
=== ZAWARTOŚĆ ŁAŃCUCHA BLOCKCHAIN ===
{
  "index": 1,
  "timestamp": 0,
  "transactions": [],
  "nonce": 0,
  "previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
--------------------------------------------------------------------
{
  "nonce": 0,
  "index": 2,
  "timestamp": 1791442507.9849508,
  "transactions": [
    "Alice -> Bob: 5"
  ],
  "previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172"
}
hash: bb3f2649ee555f3182e3e195760523703f44efaf9e4018fdc5feeef3da1bd50b
--------------------------------------------------------------------
{
  "nonce": 0,
  "index": 3,
  "timestamp": 1791442507.985005,
  "transactions": [
    "Bob -> Carol: 2"
  ],
  "previous_hash": "bb3f2649ee555f3182e3e195760523703f44efaf9e4018fdc5feeef3da1bd50b"
```



Zmodyfikowano ręcznie kwotę transakcji w bloku nr 2 i pokazano, że metoda wykrywa manipulację.

```python
        #test integralności

        is_valid, error_block_index = bc.is_chain_valid()
        print(f"Czy łańcuch blockchain jest poprawny? -> {is_valid}")
        print(f"Numer pierwszego błędnego bloku: -> {error_block_index}")

        #ręczna modyfikacja
        bc.chain[1]["transactions"][0] = "Alice -> Bob: 500"
        print("-" * 60)

        #test po modyfikacji
        valid, err_idx = bc.is_chain_valid()
        print(f"Czy łańcuch nadal jest poprawny? -> {valid} (Wykryto błąd w bloku: {err_idx})")
        print("-" * 60)
```

```
  ],
  "previous_hash": "e3b39c340ed25d8808f9edde9f262b34ff38f4e3930fa2f9e49fcd05aa654423"
}
hash: 9489d85fba4439c18a3c718343640372ac6cb8730e35f247545baefb6926c93e
------------------------------------------------------------------------
{
  "nonce": 0,
  "index": 6,
  "timestamp": 1791443366.4568932,
  "transactions": [
    "Alice -> Bob: 7"
  ],
  "previous_hash": "9489d85fba4439c18a3c718343640372ac6cb8730e35f247545baefb6926c93e"
}
hash: c5245a808d367ce80c27024857a4a368a46443807f6986763d3894b3b6a2ce03
------------------------------------------------------------------------
Czy łańcuch blockchain jest poprawny? -> True
Numer pierwszego błędnego bloku: -> None
------------------------------------------------------------
Czy łańcuch nadal jest poprawny? -> False (Wykryto błąd w bloku: 3)
------------------------------------------------------------

Process finished with exit code 0
```

Manipulacja polegała na zmianie samej treści transakcji w bloku nr 2 bez ponownego przeliczenia jego skrótu oraz bez aktualizacji pól w blokach kolejnych. W efekcie, podczas weryfikacji bloku nr 3, zapisana w nim statyczna wartość previous_hash okazała się niespójna z dynamicznie obliczonym skrótem SHA-256 zmodyfikowanego bloku nr 2. Dochodzi do naruszenia spójności struktury, co dowodzi, że jakakolwiek retroaktywna próba fałszowania danych bez jednoczesnego przeliczenia całego łańcucha zostanie natychmiast wykryta

Następnie „naprawiono” łańcuch w imieniu atakującego: przeliczono i nadpisano pola previous_hash we wszystkich blokach następujących po zmodyfikowanym.

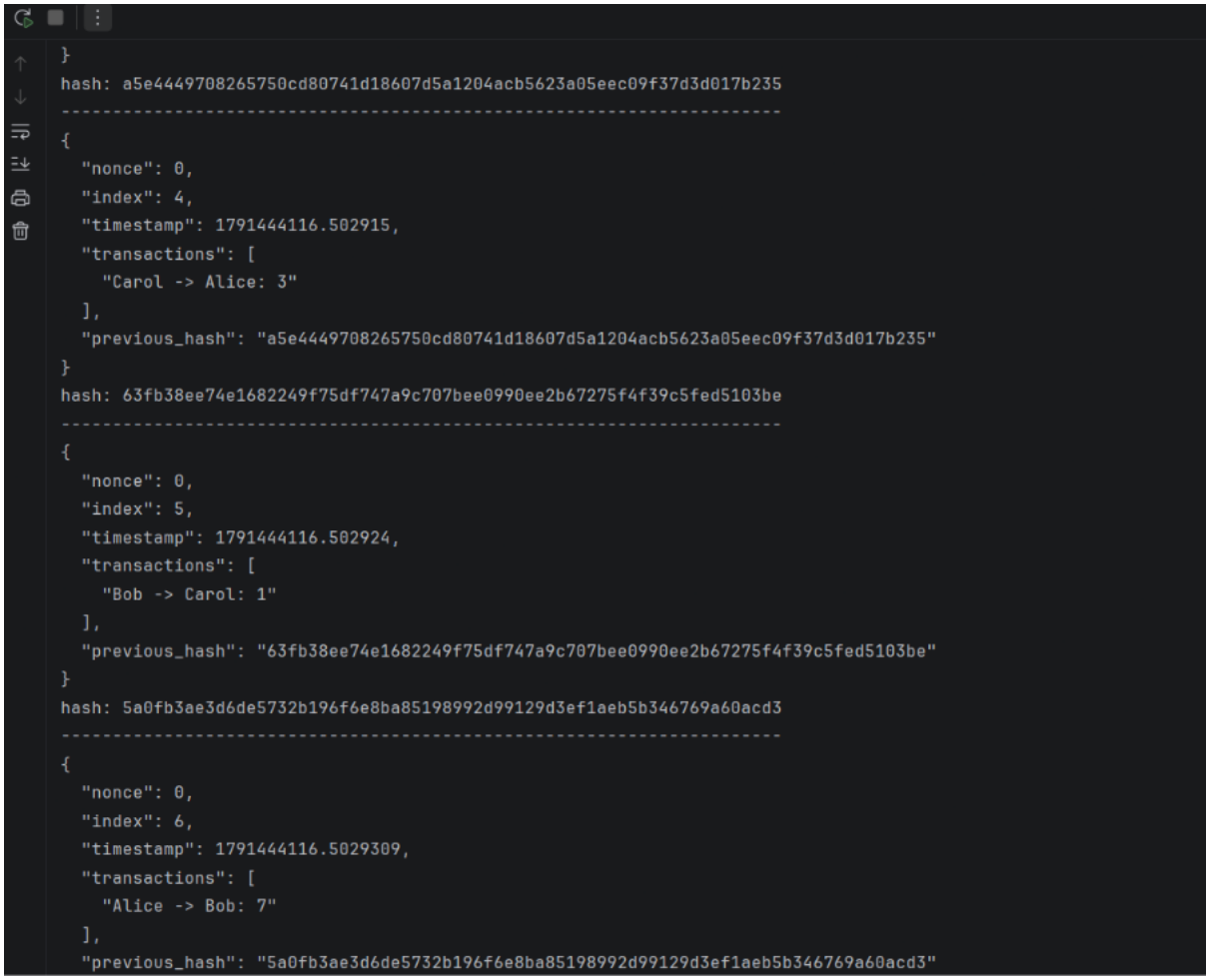
```python
    #test po modyfikacji
    valid, err_idx = bc.is_chain_valid()
    print(f"Czy łańcuch nadal jest poprawny? -> {valid} (Wykryto błąd w bloku: {err_idx})")
    print("-" * 60)

    #nowy nadpis petla i test
    for i in range(2, len(bc.chain)):
        bc.chain[i]["previous_hash"] = bc.hash(bc.chain[i - 1])

    valid, err_idx = bc.is_chain_valid()
    print(f"Czy po wielokrotnej 'naprawie' łańcuch jest poprawny? -> {valid}")
    print(f"Wykryto błąd w bloku: -> {err_idx}")
    print("-" * 60)
```

Run main

```
C:\Users\karol\OneDrive\Desktop\blockchain-lab\.venv\Scripts\python.exe C:\Users\karol\OneDrive\Desktop\blockchain-lab\main.py
=== ZAWARTOŚĆ ŁAŃCUCHA BLOCKCHAIN ===
{
  "index": 1,
  "timestamp": 0,
  "transactions": [],
  "nonce": 0,
  "previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
------------------------------------------------------------------------
{
  "nonce": 0,
  "index": 2,
  "timestamp": 1791444116.5028243,
  "transactions": [
    "Alice -> Bob: 5"
  ],
  "previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172"
}
hash: febac4d401809db847ecd3de5095a8fd4ca4fe8c3f1741cbe552170a2019b508
------------------------------------------------------------------------
{
  "nonce": 0,
  "index": 3,
  "timestamp": 1791444116.5028932,
  "transactions": [
    "Bob -> Carol: 2"
  ],
  "previous_hash": "febac4d401809db847ecd3de5095a8fd4ca4fe8c3f1741cbe552170a2019b508"
}
hash: a5e4449708265750cd80741d18607d5a1204acb5623a05eec09f37d3d017b235
```



],
"previous_hash": "5a0fb3ae3d6de5732b196f6e8ba85198992d99129d3ef1aeb5b346769a60acd3"
}
hash: 2426bda030b8d7a32c8b7962fd8b590824330ee1992d47a2567db8882a34b42d

```
Czy łańcuch blockchain jest poprawny? -> True
Numer pierwszego błędnego bloku: -> None
------------------------------------------------------------
Czy łańcuch nadal jest poprawny? -> False (Wykryto błąd w bloku: 3)
------------------------------------------------------------
Czy po wielokrotnej 'naprawie' łańcuch jest poprawny? -> True
Wykryto błąd w bloku: -> None
------------------------------------------------------------

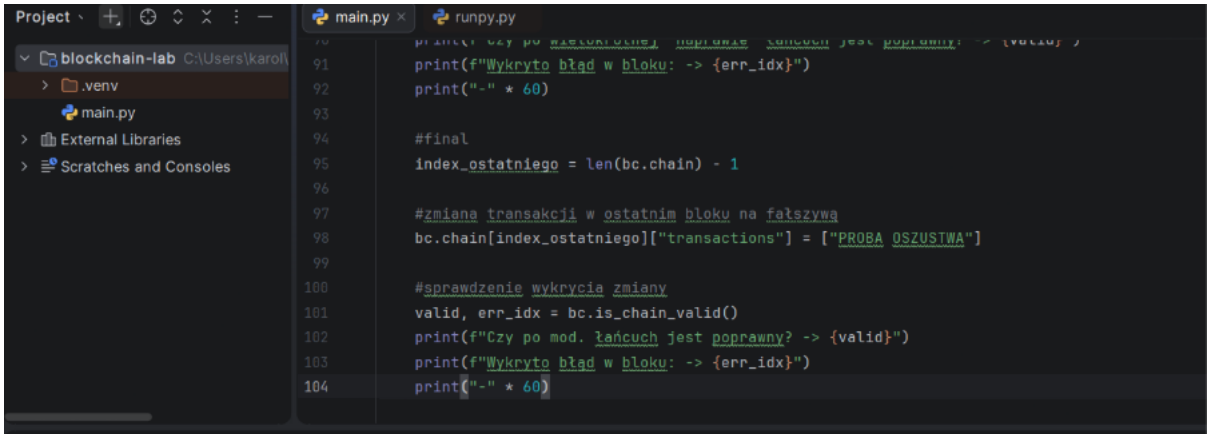
Process finished with exit code 0
```

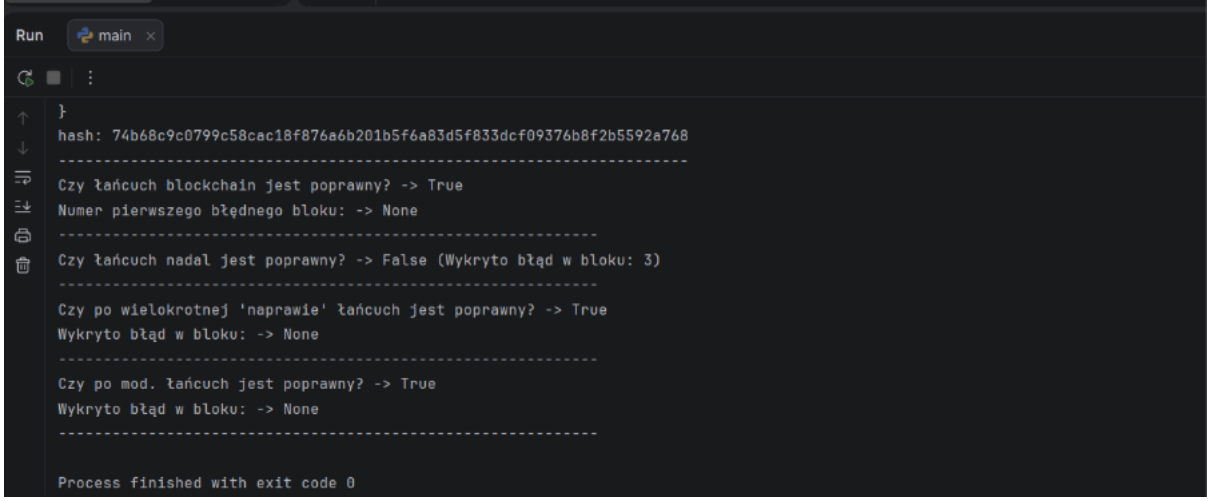
Gdy haker za pomocą pętli naprawił pole previous_hash w bloku 3, zmieniła się zawartość bloku 3. Naprawił więc blok 4, ale wtedy zmieniła się zawartość bloku 4... i tak dalej, aż do samego końca.

Gdy pętla hakera dojechała do ostatniego bloku (w tym wypadku nr 6) i zmieniła w nim previous_hash, całkowitej zmianie uległ ostateczny hash bloku nr 6.

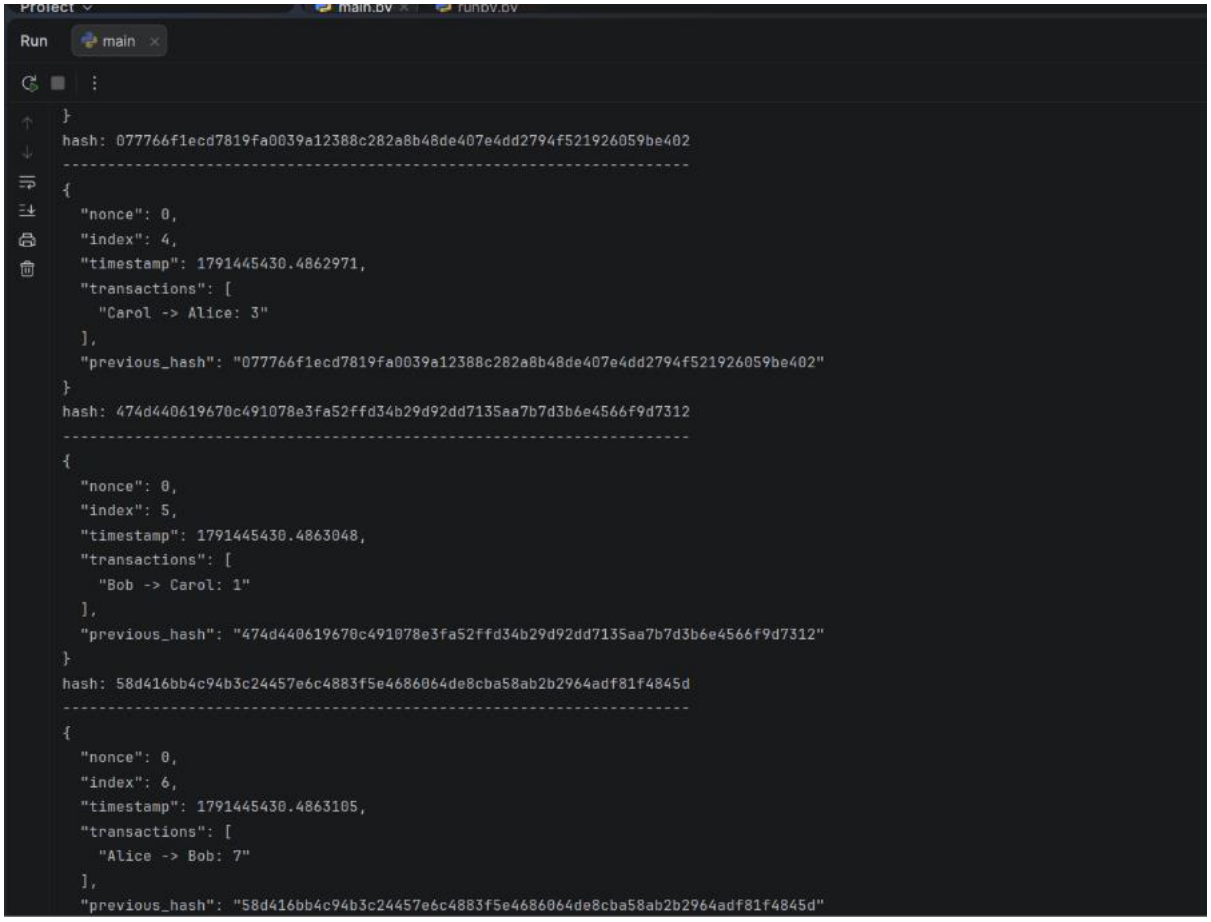
W tym momencie łańcuch pod kątem czysto wewnętrznym wydaje się spójny, ale metoda wykrywa atak, ponieważ końcowy stan całego blockchaina nie zgadza się z oficjalnym, zaufanym hashem sieci. W prawdziwym blockchainie inne komputery (węzły) odrzuciłyby taki łańcuch, ponieważ jego końcowy hash jest zupełnie inny niż ten, który posiadają wszyscy pozostali uczestnicy sieci.

Ostatnim zadaniem była modyfikacja ostatniego bloku i sprawdzenie, czy metoda to wykryje.


Project
blockchain-lab C:\Users\karol\
.venv
main.py
External Libraries
Scratches and Consoles
main.py
runpy.py
print(f"Wykryto błąd w bloku: -> {err_idx}")
print("-" * 60)
#final
index_ostatniego = len(bc.chain) - 1
#zmiana transakcji w ostatnim bloku na fałszywą
bc.chain[index_ostatniego]["transactions"] = ["PROBA OSZUSTWA"]
#sprawdzenie wykrycia zmiany
valid, err_idx = bc.is_chain_valid()
print(f"Czy po mod. łańcuch jest poprawny? -> {valid}")
print(f"Wykryto błąd w bloku: -> {err_idx}")
print("-" * 60)


Run
main
}
hash: 74b68c9c0799c58cac18f876a6b201b5f6a83d5f833dcf09376b8f2b5592a768
Czy łańcuch blockchain jest poprawny? -> True
Numer pierwszego błędnego bloku: -> None
Czy łańcuch nadal jest poprawny? -> False (Wykryto błąd w bloku: 3)
Czy po wielokrotnej 'naprawie' łańcuch jest poprawny? -> True
Wykryto błąd w bloku: -> None
Czy po mod. łańcuch jest poprawny? -> True
Wykryto błąd w bloku: -> None
Process finished with exit code 0


Run
main
C:\Users\karol\OneDrive\Desktop\blockchain-lab\.venv\Scripts\python.exe C:\Users\karol\OneDrive\Desktop\blockchain-lab\main.py
=== ZAWARTOŚĆ ŁAŃCUCHA BLOCKCHAIN ===
{
"index": 1,
"timestamp": 0,
"transactions": [],
"nonce": 0,
"previous_hash": "0000000000000000000000000000000000000000000000000000000000000000"
}
hash: 147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172
{
"nonce": 0,
"index": 2,
"timestamp": 1791445430.486224,
"transactions": [
"Alice -> Bob: 5"
],
"previous_hash": "147aed30e21a13f96f72aab4f74ca20fc513b80e10ad6caa74db670786640172"
}
hash: 84abaa17f271dfff7972608471b5f796be023e32cdc7ab68340af7b0913a7d5b
{
"nonce": 0,
"index": 3,
"timestamp": 1791445430.4862797,
"transactions": [
"Bob -> Carol: 2"
],
"previous_hash": "84abaa17f271dfff7972608471b5f796be023e32cdc7ab68340af7b0913a7d5b"
}
hash: 077766f1ecd7819fa0039a12388c282a8b48de407e4dd2794f521926059be402



],
"previous_hash": "58d416bb4c94b3c24457e6c4883f5e4686064de8cba58ab2b2964adf81f4845d"
}
hash: 74b68c9c0799c58cac18f876a6b201b5f6a83d5f833dcf09376b8f2b5592a768

Czy łańcuch blockchain jest poprawny? -> True
Numer pierwszego błędnego bloku: -> None

Czy łańcuch nadal jest poprawny? -> False (Wykryto błąd w bloku: 3)

Czy po wielokrotnej 'naprawie' łańcuch jest poprawny? -> True
Wykryto błąd w bloku: -> None

Czy po mod. łańcuch jest poprawny? -> True
Wykryto błąd w bloku: -> None

Process finished with exit code 0

Metoda is_chain_valid() nie wykryła manipulacji (zwróciła True), ponieważ algorytm weryfikuje integralność wyłącznie poprzez badanie powiązań wstecznych między dwoma sąsiadującymi blokami. Ponieważ zmodyfikowany blok był ostatnim elementem w łańcuchu, w systemie nie istnieje żaden kolejny blok, który sprawdzałby jego poprawność w swoim polu

previous_hash. Pętla zakończyła sprawdzanie przed zweryfikowaniem skrótu ostatecznego elementu.

Wnioski z przeprowadzonych ćwiczeń:

Przeprowadzone ćwiczenie laboratoryjne pozwoliło na praktyczne zrozumienie oraz implementację fundamentalnych struktur danych i mechanizmów kryptograficznych leżących u podstaw technologii blockchain. Logika powiązań oparta na funkcji skrótu SHA-256 udowodniła swoją skuteczność w zabezpieczaniu spójności danych – każda próba retroaktywnej modyfikacji transakcji wewnątrz łańcucha została natychmiast wykryta przez zaimplementowaną metodę walidacyjną. Eksperyment z pętlą „naprawiającą" oraz modyfikacją ostatniego elementu ujawnił jednak, że sam algorytm hashowania bez dodatkowych zabezpieczeń nie jest w pełni odporny na zorganizowane ataki fałszerskie. Pokazuje to, jak kluczową rolę w architekturze systemów rozproszonych odgrywają zaawansowane mechanizmy, takie jak Proof of Work oraz protokoły konsensusu sieciowego, które chronią historię transakcji przed całkowitym nadpisaniem przez intruza. Podsumowując, determinizm procesu serializacji (zapewniony przez flagę sort_keys=True) oraz matematyczna nieodwracalność skrótów stanowią niezastąpiony fundament pod budowę zdecentralizowanych i bezpiecznych rejestrów cyfrowych.

Pytania kontrolne i odpowiedzi:

1. Jakie właściwości funkcji skrótu decydują o bezpieczeństwie łańcucha bloków?

Jednokierunkowość, Odporność na kolizje oraz efekt lawinowy.

2. Dlaczego przy obliczaniu skrótu bloku stosuje się serializację z posortowanymi kluczami?

Celem zapewnienia determinizmu. W Pythonie kolejność kluczy w słownikach bywa zmienna, a algorytm SHA-256 jest skrajnie czuły na ułożenie tekstu. Sortowanie gwarantuje, że te same dane wejściowe zawsze wygenerują identyczny hash na każdym komputerze w sieci.

3. Czym różni się blok genezy od pozostałych bloków i dlaczego musi być identyczny we wszystkich węzłach?

Blok genezy to pierwszy element w łańcuchu. W przeciwieństwie do pozostałych bloków nie posiada on swojego poprzednika, dlatego jego pole previous_hash jest na stałe wpisane w kodzie jako ciąg samych zer ("0" * 64). Musi być identyczny, ponieważ każdy kolejny blok buduje swój hash w oparciu o hash bloku poprzedniego, blok genezy jest fundamentem całego łańcucha. Jeśli jakikolwiek węzeł miałby inną zawartość bloku genezy, wygenerowałby dla niego inny skrót, co spowodowałoby, że wszystkie kolejne bloki w tym

węźle miałyby zupełnie inne wartości previous_hash niż reszta sieci. Taki węzeł zostałby natychmiast odrzucony przez resztę społeczności jako niekompatybilny.

4. Dlaczego sam łańcuch skrótów nie chroni przed atakującym, który przeliczy wszystkie kolejne bloki?

Ponieważ samo przeliczenie hashu SHA-256 na współczesnym komputerze zajmuje ułamki sekund. Jeśli haker podmieni dane w bloku nr 2, a następnie użyje pętli , może w mgnieniu oka zaktualizować pola previous_hash w blokach kolejnych tworząc idealnie spójny lokalny łańcuch. Sam algorytm hashowania nie chroni przedtego rodzaju atakiem – dlatego w prawdziwych systemach blockchain stosuje się dodatkowe mechanizmy, takie jak Proof of Work (który sztucznie drastycznie spowalnia proces tworzenia i przeliczania bloków, przec co atak staje się kosztowny) oraz mechanizm konsensusu sieciowego (większość niezależnych węzłów i tak odrzuci wersję historii przedstawioną przez oszusta).